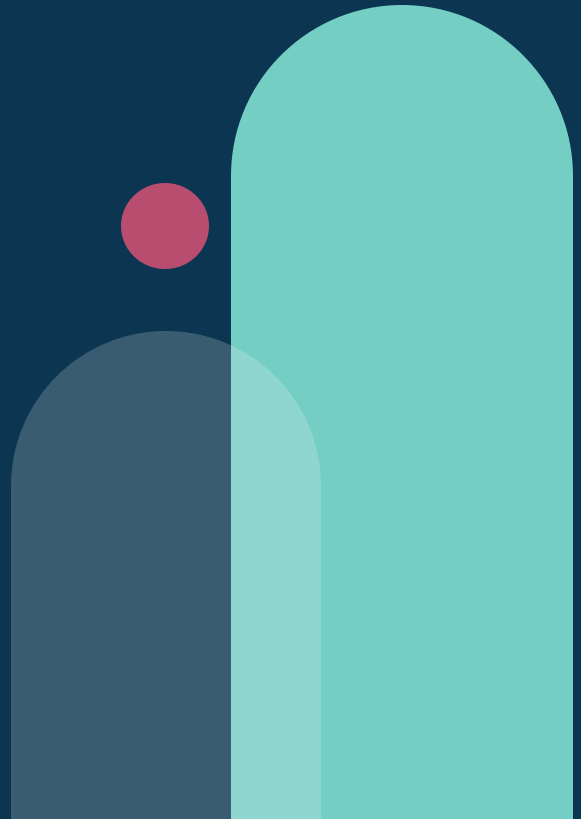
Inference & Fine-Tuning LLMs

01

Inference



LLM Inference

Inference is the process of generating predictions or completing tasks using a pre-trained model.

Common tasks: Text completion, question answering, translation, code generation.

Inference happens **after the model is trained** and involves feeding **new data** to the model to get results.

Key challenge: Balancing model size and speed of predictions.

Real-world Applications: LLM inference powers AI-based solutions like chatbots, virtual assistants, content generation, and more.

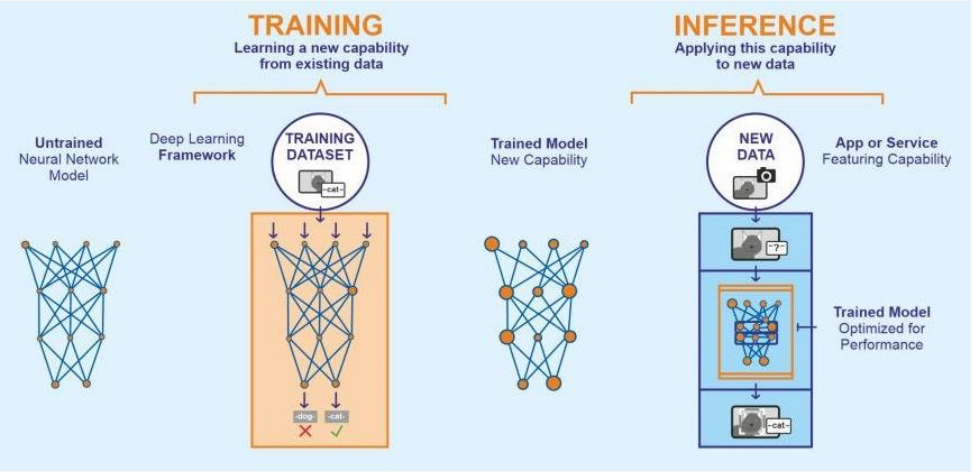


Inference vs Training

Training:
Models are built and improved by learning from large datasets.
Time-consuming and resource-intensive (days to weeks).

Inference:
The model uses the learned parameters to make predictions.
Real-time or near-real-time responses.

Key Differences:
Training updates the model; inference uses the model.
Training is done once or periodically, while inference happens every time a prediction is needed.

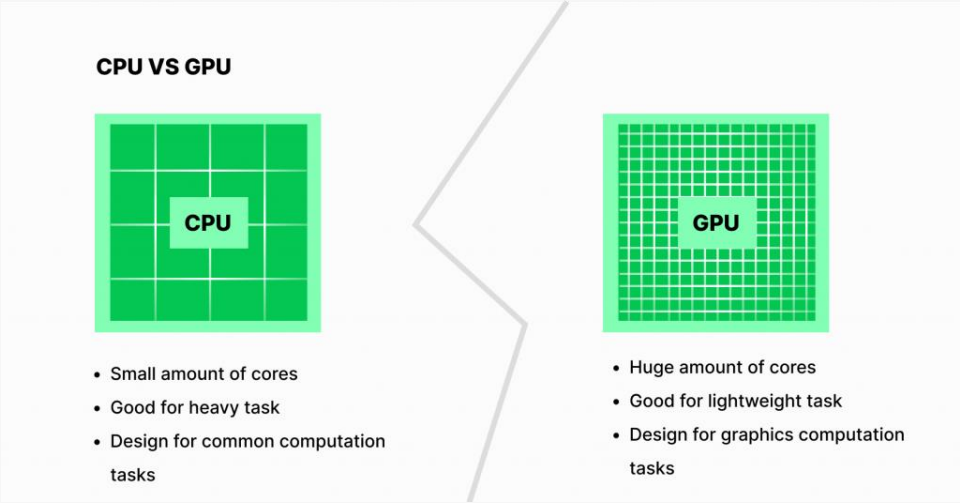


CPU vs GPU

CPU (Central Processing Unit):
Benefits: More cost-effective for small models, easier to scale across many machines.
Drawbacks: Slower for large models.

GPU (Graphics Processing Unit):
Benefits: Optimized for parallel processing, much faster for large models.
Drawbacks: Higher cost, more complex infrastructure.

Choosing Between CPU and GPU: Depends on model size, real-time requirements, and cost constraints.



VRAM Requirements and Model Size

VRAM (Video RAM): Required for loading and running models on GPUs.

Memory Usage: The larger the model, the more VRAM is required for efficient inference.

Bert-base-uncased-0.1B	0.5 GB
GPT-2-0.1B	0.56 GB
TinyLlama-1.1B-Chat	4.6 GB
Microsoft Phi-2	11.8 GB
Mistral-7B-v0.2	33 GB
Llama-3-8B-Instruct	33 GB
Llama-3-70B-Instruct	311 GB
Qwen2-72B-Instruct	322 GB
Mistral-8×22B-v0.1	631 GB
Bloom-176B	787 GB

Assumptions: Sequence Length - 512, Batch Size - 8 and Number of GPUs - 1

Local (On-Premise) vs Cloud

Local (On-Premise) Inference:

Runs on on-premise hardware, offering low latency and full data control.

Challenges: High initial hardware costs, maintenance, and limited scalability.

Use Cases: Suitable for sensitive data industries (e.g., healthcare, finance).

Cloud-based Inference:

Uses cloud platforms (AWS, Azure, GCP) with scalable infrastructure.

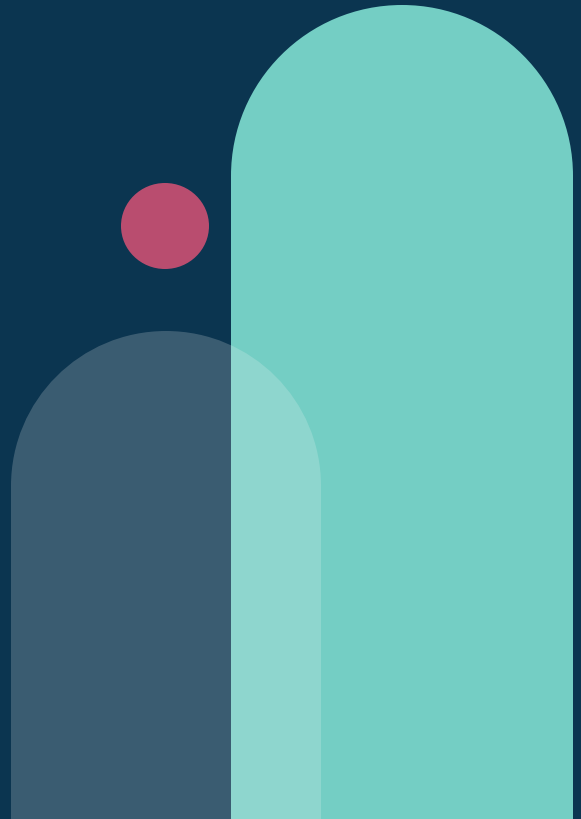
Benefits: Easy scaling, reduced hardware management, but may have higher latency due to network overhead.

Considerations: Ideal for businesses with fluctuating demand and the need for cost-effective scaling.



02

Quantization



Quantization

Quantization reduces the precision of model weights and activations from floating-point to lower precision (e.g., Float32 -> Float16).

Goal: Minimize model size and computation requirements while maintaining acceptable accuracy.

Benefits: Faster inference, reduced memory usage, and lower power consumption.



Precision Levels

INT8: 8-bit integer precision.

Commonly used for efficient inference with minimal accuracy loss.

INT4: 4-bit integer precision.

Maximizes efficiency but can cause noticeable accuracy degradation in sensitive models.

Float16 (FP16): 16-bit floating-point precision.

Strikes a balance between speed and precision, used in many deep learning accelerators.

Float32 (FP32): 32-bit floating-point precision.

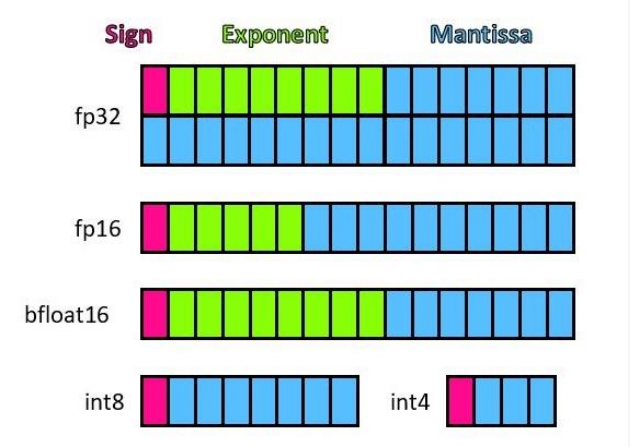
Standard precision used during model training, but computationally expensive for inference.

BF16 (BFloat16): 16-bit floating-point format with higher dynamic range than FP16.

Used to maintain precision in large models during training while improving performance.

Mixed Precision: Combines lower precision (e.g., FP16 or BF16) with higher precision (e.g., FP32) in different parts of the model.

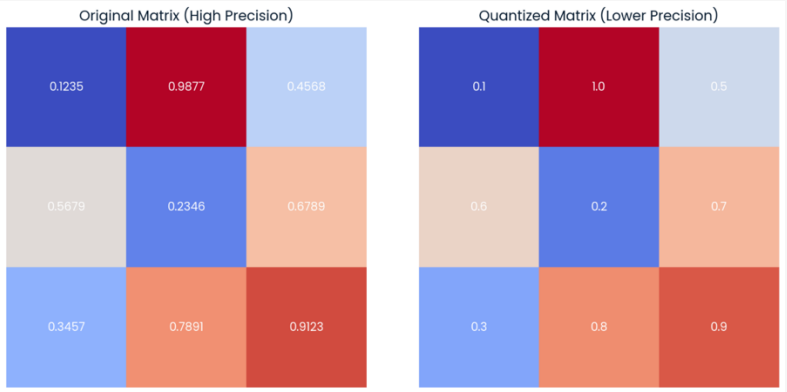
Optimizes both memory usage and speed without significantly affecting accuracy.



Quantization

Quantization is normally applied to the weights of a large language model, although it can also be applied to the activations.

Weights are essentially the learned coefficients that transform input data as it passes through the network.



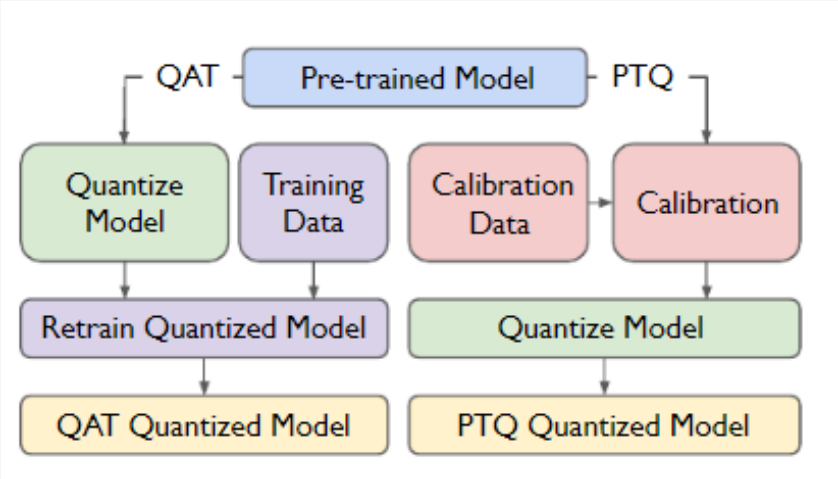
Types of Quantization

Post-training Quantization (PTQ):

- Applied after the model is trained.
- Simplest method, reduces model size with minimal accuracy drop.

Quantization-Aware Training (QAT):

- Quantization is simulated during training.
- Yields better accuracy for sensitive models as the model adapts to quantization effects during training.



GGUF and Quantization methods

GGUF (Previously GGML):

• A unified format for storing full and quantized versions of LLM models, designed for efficient inference on diverse hardware setups.

• Enables loading large models into memory-efficient formats without significant loss of performance.

Easy to download from HuggingFace and run it using llama.cpp, lmstudio, ollama, gpt4all etc

Filename	Quant type	File Size	Split	Description
Meta-Llama-3.1-8B-Instruct-f32.gguf	f32	32.13GB	false	Full F32 weights.
Meta-Llama-3.1-8B-Instruct-Q8_0.gguf	Q8_0	8.54GB	false	Extremely high quality, generally unneeded but max available quant.
Meta-Llama-3.1-8B-Instruct-Q6_K_L.gguf	Q6_K_L	6.85GB	false	Uses Q8_0 for embed and output weights. Very high quality, near perfect, *recommended*.
Meta-Llama-3.1-8B-Instruct-Q6_K.gguf	Q6_K	6.60GB	false	Very high quality, near perfect, *recommended*.
Meta-Llama-3.1-8B-Instruct-Q5_K_L.gguf	Q5_K_L	6.06GB	false	Uses Q8_0 for embed and output weights. High quality, *recommended*.
Meta-Llama-3.1-8B-Instruct-Q5_K_M.gguf	Q5_K_M	5.73GB	false	High quality, *recommended*.
Meta-Llama-3.1-8B-Instruct-Q5_K_S.gguf	Q5_K_S	5.60GB	false	High quality, *recommended*.
Meta-Llama-3.1-8B-Instruct-Q4_K_L.gguf	Q4_K_L	5.31GB	false	Uses Q8_0 for embed and output weights. Good quality, *recommended*.
Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf	Q4_K_M	4.92GB	false	Good quality, default size for must use cases, *recommended*.
Meta-Llama-3.1-8B-	Q3_K_XL	4.78GB	false	Uses Q8_0 for embed and output

VRAM for Quantized LLMs

$$M = \frac{(P * 4B)}{(32/Q)} * 1.2$$

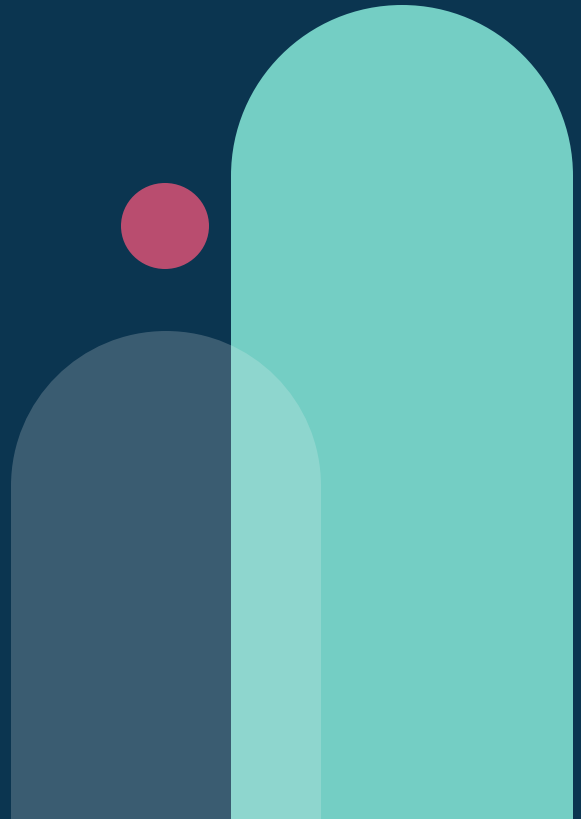
Symbol	Description
M	GPU memory expressed in Gigabyte
P	The amount of parameters in the model. E.g. a 7B model has 7 billion parameters.
4B	4 bytes, expressing the bytes used for each parameter
32	There are 32 bits in 4 bytes
Q	The amount of bits that should be used for loading the model. E.g. 16 bits, 8 bits or 4 bits.
1.2	Represents a 20% overhead of loading additional things in GPU memory.

	FP32	FP16	INT8
Bert-base-uncased-0.1B	0.5 GB	0.24 GB	0.12 GB
GPT-2-0.1B	0.56 GB	0.28 GB	0.14 GB
TinyLlama-1.1B-Chat	4.6 GB	2.3 GB	1.1GB
Microsoft Phi-2	11.8 GB	6 GB	3 GB
Mistral-7B-v0.2	33 GB	16.5 GB	8.25 GB
Llama-3-8B-Instruct	33 GB	17 GB	8.5GB
Llama-3-70B-Instruct	311 GB	155 GB	77 GB
Qwen2-72B-Instruct	322 GB	161 GB	80 GB
Mistral-8×22B-v0.1	631 GB	316 GB	158 GB
Bloom-176B	787 GB	394 GB	200 GB

https://rahulschand.github.io/gpu_poor/
https://llm-system-requirements.streamlit.app/

03

Fine-Tuning LLMs



What is LLM Fine-Tuning?

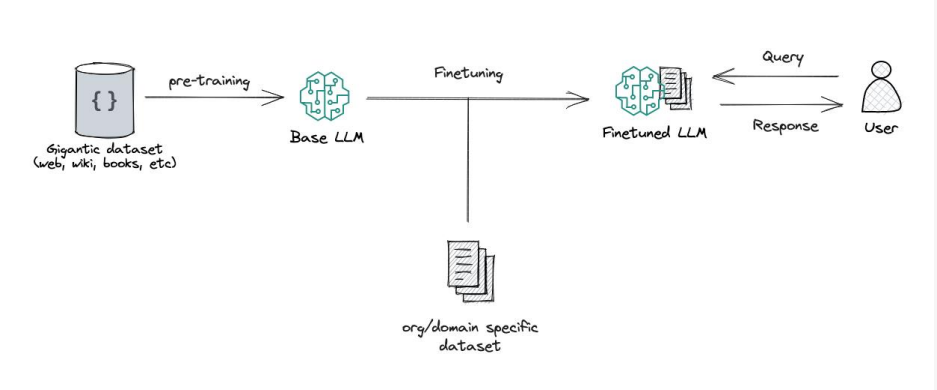
Fine-tuning involves additional training of a pre-trained LLM using a smaller, domain-specific dataset.

It adapts the model to perform better on specific tasks or within particular domains.

Benefits:

- Reduces training costs and resources.
- Enables more accurate and context-specific results.

Example: Fine-tuning GPT models for specialized tasks like medical text analysis or sentiment detection.



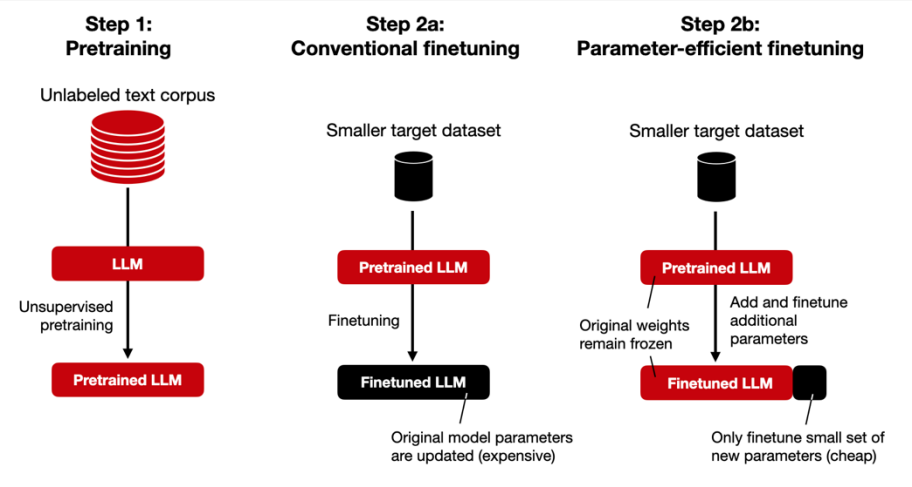
Fine-Tuning Methods

Full Fine-Tuning (Instruction Fine-Tuning):

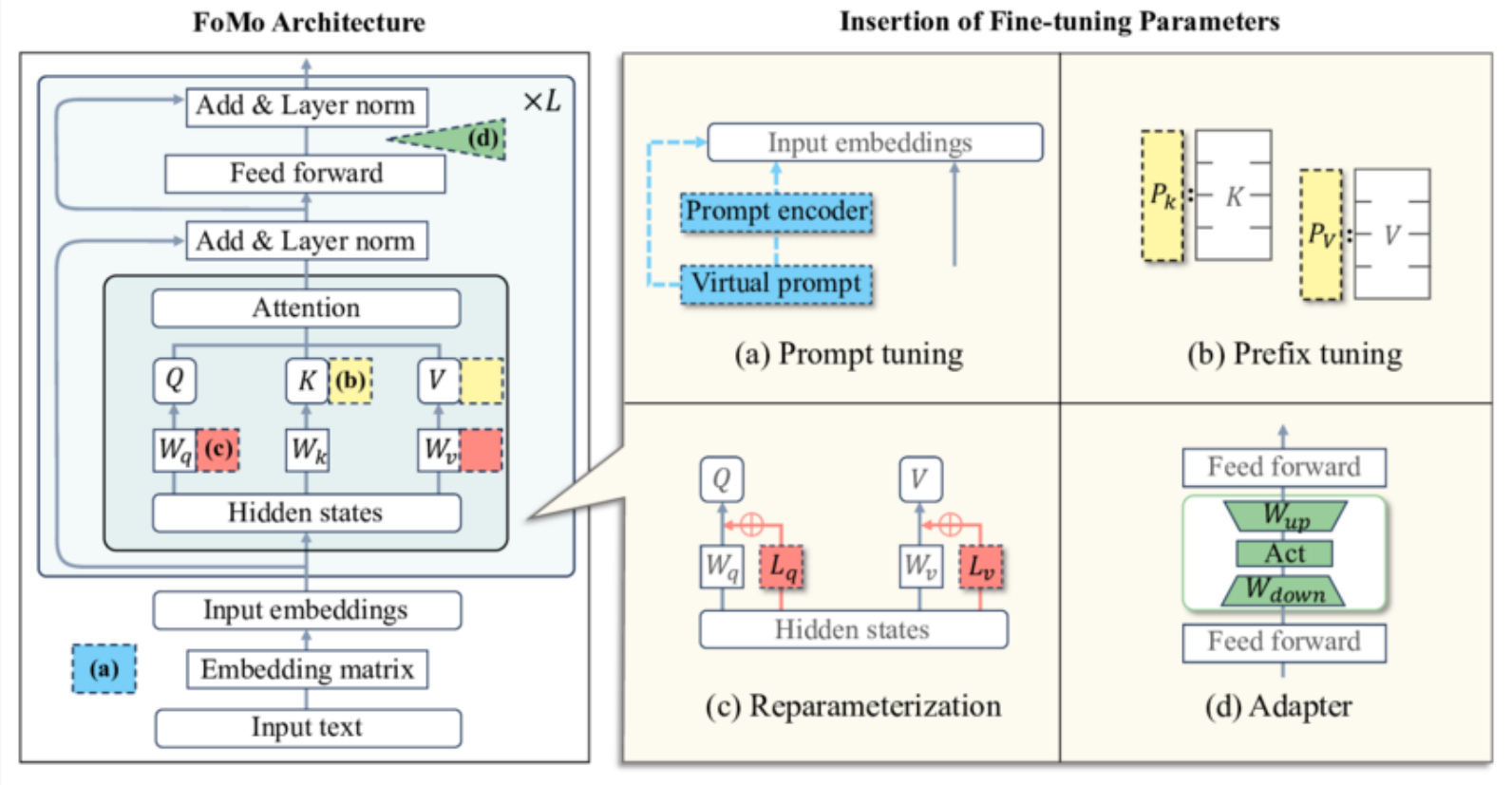
- Updates all the model's parameters.
- Requires significant memory and computational resources, similar to pre-training.
- Best for when you need comprehensive model adaptation.

Parameter-Efficient Fine-Tuning (PEFT):

- Only adds/updates a small subset of parameters, keeping most of the model weights frozen.
- Reduces memory and compute requirements, preventing "catastrophic forgetting."
- Ideal for fine-tuning large models with limited resources.



PEFT


FoMo Architecture
Add & Layer norm
×L
(d)
Feed forward
Add & Layer norm
Attention
Q
K
(b)
V
W_q
(c)
W_k
W_v
Hidden states
Input embeddings
(a)
Embedding matrix
Input text
Insertion of Fine-tuning Parameters
Input embeddings
Prompt encoder
Virtual prompt
(a) Prompt tuning
P_k
K
P_v
V
(b) Prefix tuning
Q
V
W_q
L_q
W_v
L_v
Hidden states
(c) Reparameterization
Feed forward
W_{up}
Act
W_{down}
Feed forward
(d) Adapter

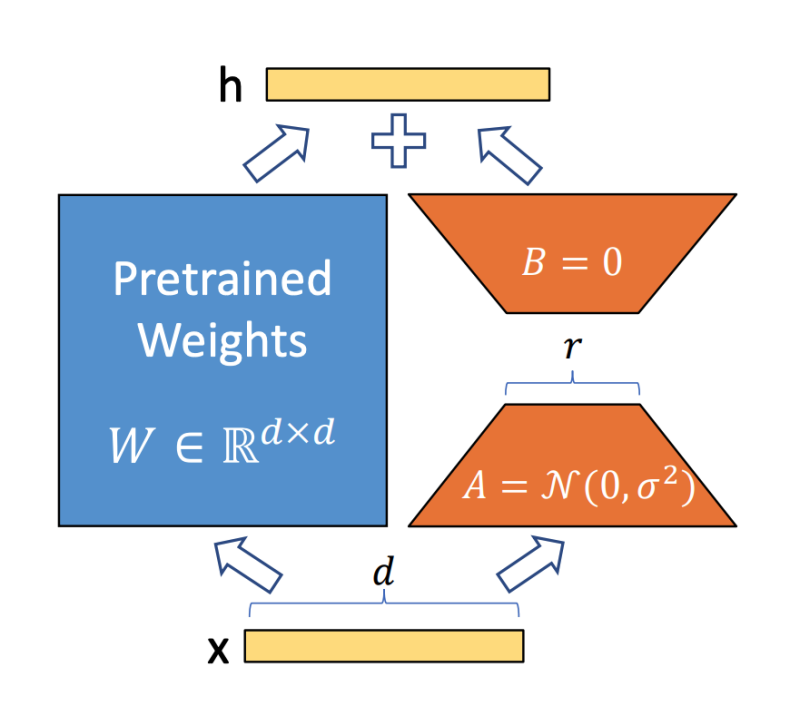
LoRA

LoRA fine-tunes two smaller matrices that approximate the larger weight matrix of an LLM.

Advantages:

- Significantly reduces memory requirements.
- The original LLM remains unchanged.
- LoRA adapters are much smaller (MBs vs. GBs) and can be reused for different tasks.

Multiple LoRA adapters can use the same base LLM for different use cases, saving on storage.



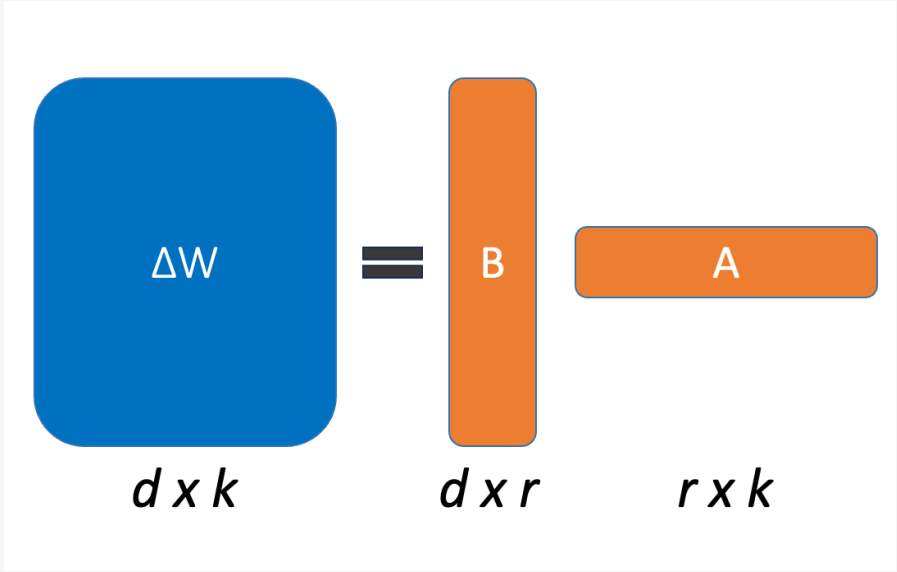
LoRA

Let:
d = 2048
k = 2048
r = 16
PreTrained:

d x k = 41,672,704

LoRA:

(d x r) + (r x k). = 32,768

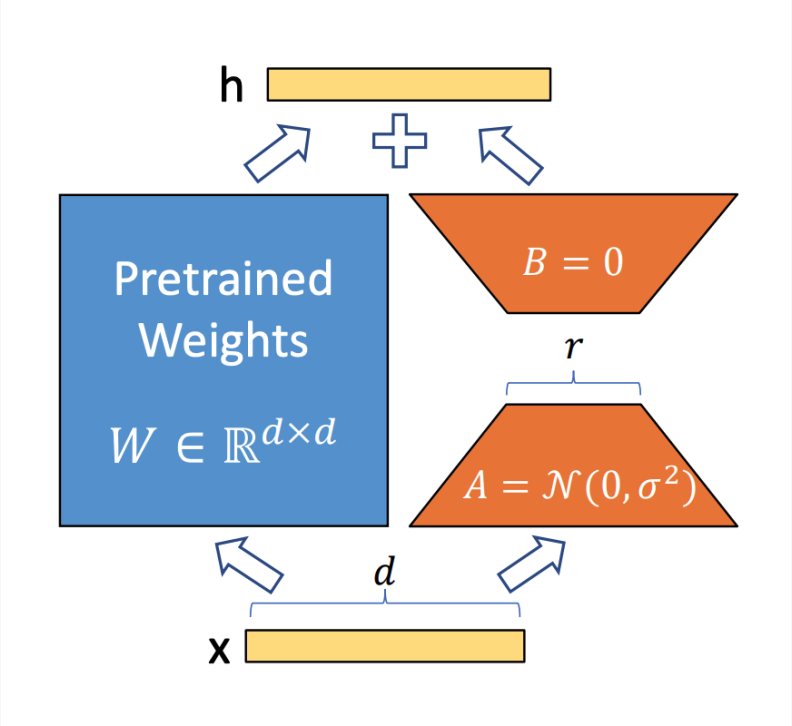


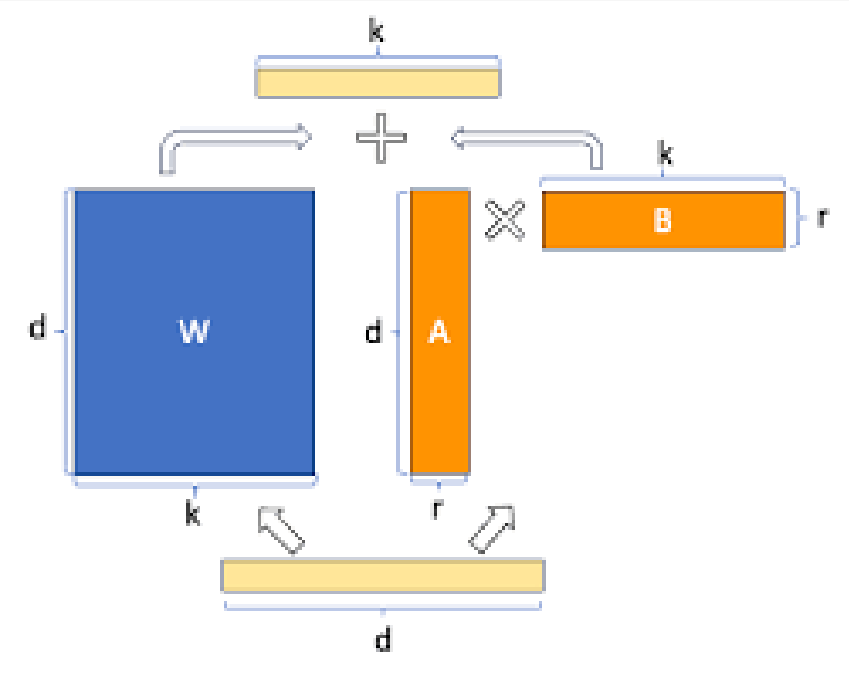
Ex:

$$\begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} \times \begin{bmatrix} 6 & 4 & 3 \end{bmatrix} = \begin{bmatrix} 12 & 8 & 6 \\ 18 & 12 & 9 \\ 24 & 16 & 12 \end{bmatrix}$$

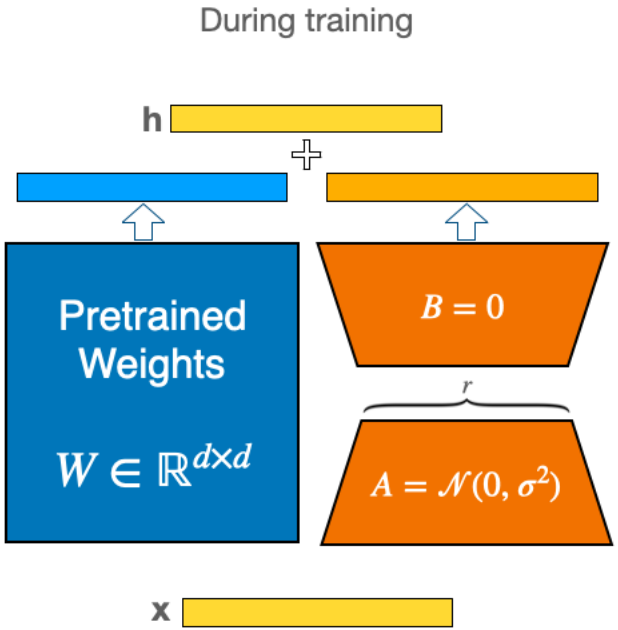
6 params projected as 9 params

LoRA


h
Pretrained Weights
$W \in \mathbb{R}^{d \times d}$
$B = 0$
r
$A = \mathcal{N}(0, \sigma^2)$
d
x

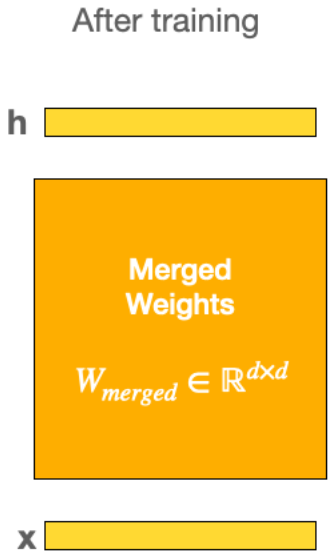

k
k
B
r
d
W
d
A
k
r
d

LoRA



$$h = Wx + BAx$$

$$h = \underbrace{(W + BA)}_{W_{merged}}x$$

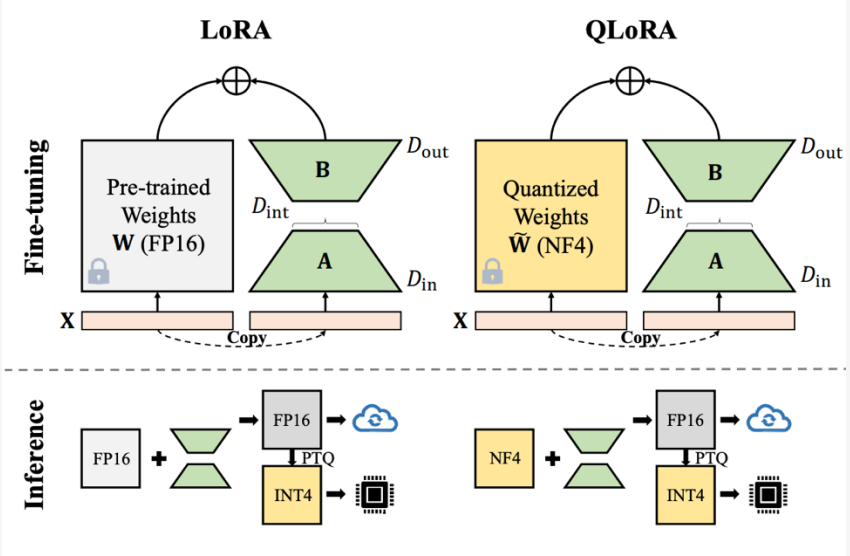


QLoRA

A more memory-efficient version of LoRA that applies 4-bit quantization to the LoRA adapters.

Benefits:

- Further reduces memory and storage requirements while maintaining model performance.
- The pre-trained model is loaded into GPU memory with 4-bit precision, instead of the standard 8-bit.
- Effective for scenarios where memory optimization is crucial, without sacrificing task-specific fine-tuning performance.



04

Fine-Tuning LLMs using LoRA

Thanks!

Do you have any questions?

